

The University of Lahore
Department of Computer Science & IT
Final Examination



Spring-2022

Course Title:	Programming Fundamentals	Course Code:	CS02102	Credit Hours:	4
Course Instructors:	Dr. Abbas Khalid Mr. Muhammad Adress, Ms. Salwa Muhammad Akhtar, Ms. Naila Rafique	Program Name:	BSCS		
Time Allowed:	120 Minutes	Maximum Marks:	40		
Date:	01-06-2022	Course Mentor:	Dr. Abbas Khalid		
Student's Name:		Signature:			
Reg. No:		Section:			

Important Instructions / Guidelines:

- Cutting, overwriting, using correction fluid is not allowed in the objective part.
- **Solve question 1, 2 on question paper and question 3 on the answer sheet provided separately.**
- Your paper will be marked as ZERO, if involved in any kind of cheating.

Question # 1: Choose the correct answer and encircle it. (10x1=10 Marks)

1.	string* x, y; means "x is a pointer to a string, y is a string" a) True b) False c) Incomplete information given d) None of these
2.	Which of the following is illegal? a) int i=4, *a; b) int i, *a; a=&i; c) int i; char* a; a=&i; d) int i=4, *a; a=&i; *a=2
3.	The location of variable in memory is passed to the function in which of the following a) Pass by reference b) Pass by value c) Pass by function d) None of these
4.	A function that is expanded/copy at call during compile time is called: a) Pass by function b) Function overloading c) Inline function d) None of these
5.	How many elements can be given to the following array: int array [5][8]; a) 5 b) 8 c) 40 d) 13
6.	Index number of the last element of an array having 9 elements is a) 9 b) 10 c) 8 d) None of these
7.	How can we access the 7th element of an int array? a) cout<<array[7]; b) cout<<array[6]; c) cout<<array[0]; d) None of these
8.	The execution of the code always begins from a) main () b) first user-defined function c) first built-in function d) None of these
9.	For int array [10], the index array [10] is a) In-bound index b) Out-of-bound index c) Both of these d) None of these
10.	Arrays can be passed to a function as a) Pass by value b) Pass by reference c) Both of these d) None of these

The University of Lahore
Department of Computer Science & IT
Final Examination

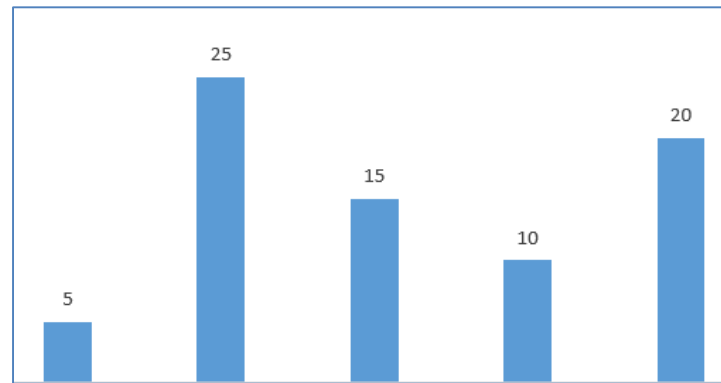
Spring-2022

CS02102-Programming Fundamentals

Question # 2: Show output or identify errors if any **(2x2.5=5 Marks)**

Code	Output
<pre>int main () { int a=10, b=20, *p1, *p2; p1 = &a; p2 = &b; cout << *p1 << " " << *p2 << endl; p1 = p2; cout << *p1 << " " << *p2 << endl; *p1 = 30; cout << *p1 << " " << *p2 << endl; *p1 = *p2; cout << *p1 << " " << *p2 << endl; return 0; }</pre>	<pre>10 20 20 20 30 30 30 30</pre>
<pre>int main () { double *fp, f1, f2; fp = &f1; *fp = 4.5; fp = &f2; *fp = 10.69; cout << f1 << " " << f2 << endl; return 0; }</pre>	<pre>4.5 10.69</pre>

- a) Assuming you have 5 towers of varying heights as given in the figure below



Your task is to sort these towers according to their heights in ascending order (the shortest tower should come first and the longest tower should come in the end). **Perform this task using 1D-Array. You can use any sorting algorithm.**

SOLUTION:

BUBBLE SORT:

```
#include<iostream>
using namespace std;
int main ()
{
    int temp, a[5] = {5,25,15,10,20};
    for(int i = 0; i<5; i++)
    {
        for(int j = i+1; j<5; j++)
        {
            if(a[j] < a[i])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
    cout <<"Sorted Array"<<endl;
    for(int i = 0; i<5; i++)
    {
        cout <<a[i]<<" ";
    }
    return 0;
}
```

SELECTION SORT:

```
#include<iostream>
using namespace std;
```

```

int main()
{
    int a[5] = {5,25,15,10,20}, temp, min, x, index;

    for(int i=0; i<5; i++)
    {
        x=0;
        min = a[i];
        for(int j=(i+1); j<5; j++)
        {
            if(min>a[j])
            {
                min = a[j];
                x++;
                index = j;
            }
        }
        if(x!=0)
        {
            temp = a[i];
            a[i] = min;
            a[index] = temp;
        }
    }
    cout<<"Sorted Array"<<endl;
    for(int i=0; i<5; i++)
    {
        cout<<a[i]<<" ";
    }
    cout<<endl;
    return 0;
}

```

- b) Given the following array, double height [10] = {2.7, 1.2, 8.8, 9.1, 5.9, 4.8, 10.8, 6.7, 9.1, 10.5}. Your task is to show how many students have a height greater than or equal to 4.6 feet.

SOLUTION:

```

#include<iostream>
using namespace std;
int main()
{
    double height [10] = {2.7, 1.2, 8.8, 9.1, 5.9, 4.8, 10.8, 6.7, 9.1, 10.5};
    int count = 0;
    for (int i=0; i<10; i++)
    {
        if (height[i]>=4.6)
        {
            count++;
            cout<<"Value is greater or equal to 4.6 at index "<<i<<endl;
        }
    }
}

```

```
cout<<"Total students having height greater or equal to 4.6 are : "<<count<<endl;
return 0;
}
```

- c) Write a C++ program to build a calculator using user-defined functions. This calculator must be menu-driven, i.e., user must be shown the options of arithmetic operations they can perform. **The selection of arithmetic operation should be done using switch or if-else statement.** Your calculator must have the following functions to perform the arithmetic operation accordingly

- add()
- sub()
- divide ()
- multiply ()
- mod ()

Sample Output:

Select one of the following + - * / %

+

Enter the two values to be added:

5

7

The sum of 5 and 7 is 12

SOLUTION:

```
#include<iostream>
using namespace std;
```

```
int add(int a, int b);
int sub(int a, int b);
int mul(int a, int b);
int div(int a, int b);
int mod(int a, int b);
```

```
int main ()
{
    char choice;
    int a,b;
    cout<<"Enter two operands : ";
    cin>>a>>b;
    cout<<"Please select an operator to perform the arithmetic operation"<<endl;
    cout<<"Options: \n + \n - \n * \n / \n % "<<endl;
    cout<<"Enter your choice : ";
    cin>>choice;
    switch (choice)
    {
        case '+':
            cout<<add(a,b);
            break;
```

```

        case '-':
            cout<<sub(a,b);
            break;
        case '*':
            cout<<mul(a,b);
            break;
        case '/':
            cout<<div(a,b);
            break;
        case '%':
            cout<<mod(a,b);
            break;
        default:
            cout<<"Invalid choice";
    }
    return 0;
}

int add (int a, int b)
{
    return a+b;
}

int sub (int a, int b)
{
    return a-b;
}

int mul (int a, int b)
{
    return a*b;
}

int div (int a, int b)
{
    return a/b;
}

int mod (int a, int b)
{
    return a%b;
}

```

- d) The absolute value of a real number x , denoted by $|x|$, is the non-negative value of x , e.g., if x is -5 , then $|x|$ will be 5 . Your task is to make function/functions named *absolute*. In your code the user will provide a negative argument (either in int or float or double datatype), and the *absolute ()* returns the corresponding absolute value.

Note: It must be noted here that if the user gives a positive value then that value is returned unchanged. Also, no typecasting (either implicit or explicit) must be done in this code.

Sample Output 1:

```

Enter value to calculate absolute value: -5
Absolute value is = 5

```

Sample Output 2:

Enter value to calculate absolute value: -9.2
Absolute value is = 9.2

Sample Output 3:

Enter value to calculate absolute value: 8
Absolute value is = 8

SOLUTION:

```
#include<iostream>
using namespace std;

int absolute (int a);
float absolute (float a);
double absolute (double a);

int main ()
{
    cout<<absolute(-4)<<endl;
    cout<<absolute(-1.2f)<<endl;
    cout<<absolute(-3.5)<<endl;
    cout<<absolute(6)<<endl;
    return 0;
}

int absolute (int a)
{
    if (a < 0)
    {
        return (-1)*a;
    }
    else
    {
        return a;
    }
}

float absolute (float a)
{
    if (a < 0)
    {
        return (-1)*a;
    }
    else
    {
        return a;
    }
}

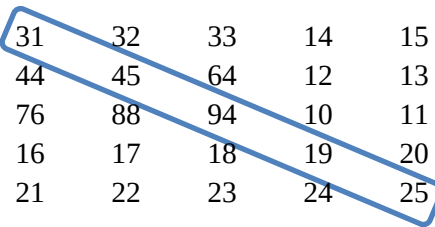
double absolute (double a)
```

```

    {
        if (a < 0)
        {
            return (-1)*a;
        }
        else
        {
            return a;
        }
    }
}

```

- e) You are given a 2D-Array of int datatype having 5 rows and 5 columns. Your task is to write a C++ program that calculates the sum of the diagonal. For example, if you have the following array then the circled part is the diagonal whose sum is to be calculated.



31	32	33	14	15
44	45	64	12	13
76	88	94	10	11
16	17	18	19	20
21	22	23	24	25

SOLUTION:

```

#include<iostream>
using namespace std;

int main ()
{
    int sum = 0;
    int a[5][5]={31,32,33,14,15,
                 44,45,64,12,13,
                 76,88,94,10,11,
                 16,17,18,19,20,
                 21,22,23,24,25};

    for (int i=0; i<5; i++)
    {
        for (int j=0; j<5; j++)
        {
            if (i==j)
            {
                sum = sum + a[i][j];
            }
        }
    }
    cout<<sum;
    return 0;
}

```